

Thread Life Cycle in C# with Examples

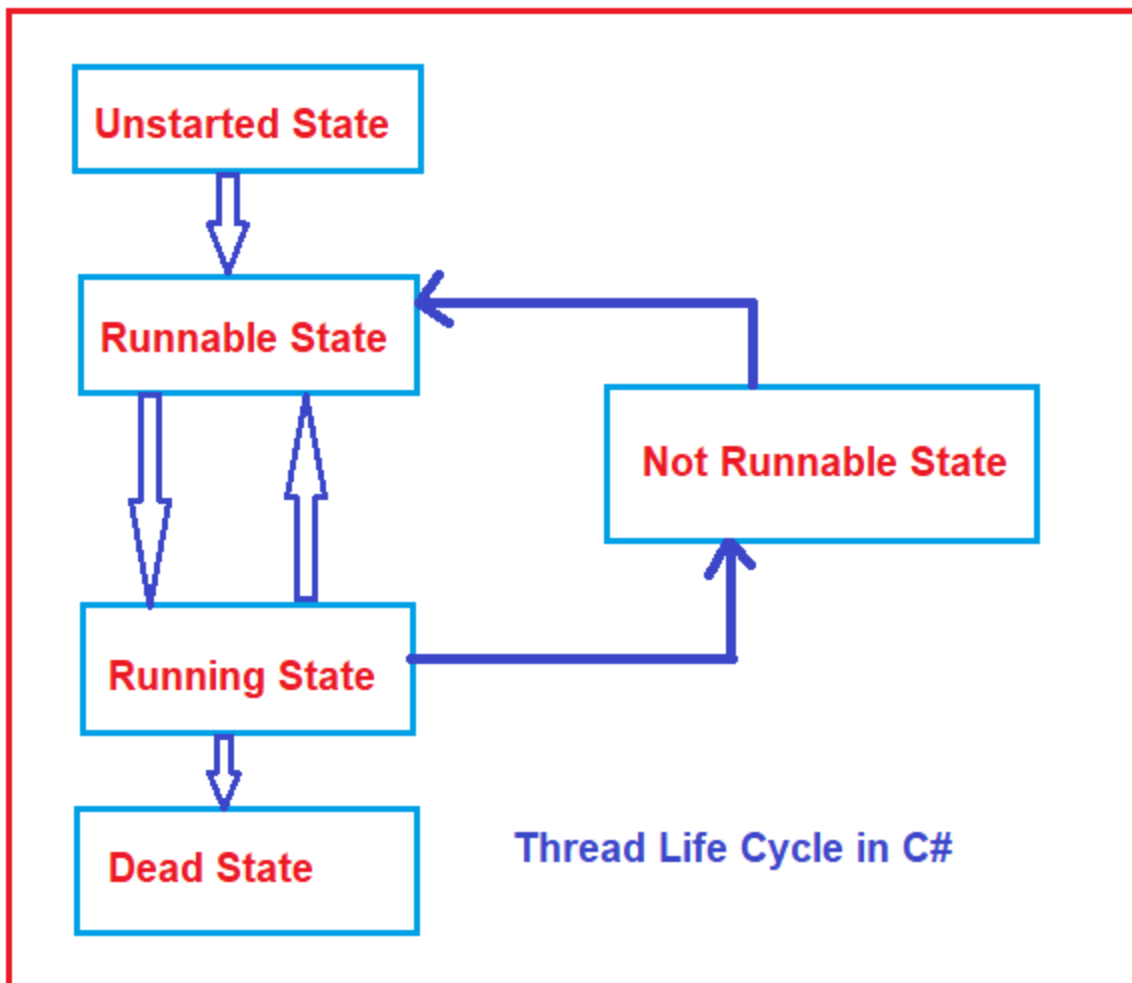
In this article, I am going to discuss **Thread Life Cycle in C#** with Examples. Please read our previous article where we discussed [AutoResetEvent and ManualResetEvent in C#](#) with Examples.

Thread Life Cycle in C#

A thread in C# has a life cycle, which will start when we create an instance of the Thread class, and the thread life cycle ends when the task execution of the thread is completed or when the thread is terminated. A thread in C# at any point in time exists in any one of the following states.

1. **Unstarted (New) State**
2. **Runnable State (Ready to Run)**
3. **Running**
4. **Not Runnable State**
5. **Dead State**

For a better understanding, please have a look at the below diagram which shows the different states of the thread in C#.



Now, let us understand each state in detail.

Unstarted State or New State:

When we create an instance of a Thread class, it is in an unstarted state. That means the thread is just created but it is not yet started. That means we have created the thread object but not called Start() method. The following is an example to create a thread instance in C#,

```
Thread thread = new Thread(SomeMethod);
```

Once you execute the above statement, the thread is created but it is in the unstarted state. In the unstarted state, a thread is not considered alive. Once the Start() method is called on the thread instance, it leaves the unstarted state and enters the next state but once it leaves the unstarted state, it is not impossible for a thread to return back to this unstarted state in its lifetime. The unstarted state can also be called the new state.

Runnable State (Ready to Run):

When we call the start() method, then the thread will move from the unstarted state to the runnable or ready-to-run state. That means now the thread is eligible to run, but it is not running yet, because the thread scheduler has not selected it to run. It might be possible that there could be multiple threads in the runnable state. If there are multiple threads in the runnable state, then the thread scheduler will decide which thread from the runnable state should be moved to the next state i.e. Running state. A thread in the runnable state is considered to be alive. A thread can return to a runnable state after coming back from sleeping, waiting/blocked, or running state.

Running State:

A thread will enter into the running state, when the thread scheduler selected it to run (out of all the threads which are ready to run or in the runnable state), only one thread within a process can be executed at a time. At the time of execution, the thread is in the running state. A thread in the Running state is alive. From the running state, a thread can enter into the Not Runnable state, Runnable State, or Dead state.

Not Runnable State:

A thread in C# enters into the Not Runnable State in the following scenarios. When a thread finds itself in any of the below situations, then the thread will move into the not runnable state and the thread is no longer eligible to run, but even in this state, the thread is still considered to be alive. Some people also refer to this state as the WaitSleepJoin state.

1. When we called the Wait() method on the thread object and it is waiting for other threads to notify it or wake it up.
2. When we called the Sleep() method on the thread object and asked it to sleep for some duration of time.
3. When a thread has called the Join() method on another thread, which makes the first thread wait until another thread has finished its execution.
4. When a thread is waiting for an Input/Output or other resources to be free.

Note: When the thread gets out of this Not Runnable state, then it will re-enter into a runnable state.

Dead State:

When the thread completed its task, the thread enters into a dead, terminates, or abort state. That means the execution of the thread is finished. This is the last state in a thread's life cycle. A thread enters the dead state

after it has successfully completed executing its entry point method i.e. Start(), or when the Abort() method has been called on it to abort its execution. In this state, a thread is considered to be not alive, and hence if you try to call Start() method on a dead thread, it raises the ThreadStateException exception.

Important Properties and Methods of Thread Class in C#:

The Thread Class in C# provides various properties that allow us to perform different tasks such as obtaining the status of a thread and specifying a name for the thread. Following are the properties of the Thread class in C#.

1. **CurrentThread**: It gets the currently running thread instance. That means it returns a Thread instance that is the currently running thread.
2. **IsAlive**: It gets a value indicating the execution status of the current thread. It returns true if this thread has been started and has not terminated normally or aborted; otherwise, false.
3. **Name**: It is used to get or set the name of the thread. It returns a string containing the name of the thread, or null if no name was set.
4. **ThreadState**: It gets a value containing the states of the current thread. It returns one of the System.Threading.ThreadState values indicate the state of the current thread. The initial value is Unstarted.

The Thread Class in C# provides the following methods to implement the states of the threads.

1. **Sleep()**: This method Suspends the current thread for the specified amount of time.
2. **Join()**: This method blocks the calling thread until the thread represented by this instance terminates while continuing to perform standard COM and SendMessage pumping.
3. **Abort()**: This method Raises a System.Threading.ThreadAbortException in the thread on which it is invoked, to begin the process of terminating the thread. Calling this method usually terminates the thread.
4. **Suspend()**: This method either suspends the thread or if the thread is already suspended, has no effect.
5. **Resume()**: This method resumes a thread that has been suspended.
6. **Start()**: This method causes the operating system to change the state of the current instance to the Running state.

Example to understand Thread Life Cycle States in C#:

The following example shows the different states of the thread. These states of the thread are determined by using the ThreadState property of the Thread class. Also, we use Suspend() and Resume() methods to suspend the current execution of the thread and resume the suspended thread by using the Resume() method. We have also called the Abort method to terminate the thread and after terminating the thread, again, we are calling the Start method on the thread instance which will throw an exception.

```
using System;
using System.Threading;
namespace ThreadStateDemo
{
    class Program
    {
        static void Main(string[] args)
        {
```

```

try
{
    // Creating and initializing threads Unstarted state
    Thread thread1 = new Thread(SomeMethod);
    Console.WriteLine($"Before Start, IsAlive: {thread1.IsAlive},
ThreadState: {thread1.ThreadState}");
    // Running State
    thread1.Start();
    Console.WriteLine($"After Start(), IsAlive: {thread1.IsAlive},
ThreadState: {thread1.ThreadState}");
    // thread1 is in suspended state
    thread1.Suspend();
    Console.WriteLine($"After Suspend(), IsAlive: {thread1.IsAlive},
ThreadState: {thread1.ThreadState}");
    // thread1 is resume to running state
    thread1.Resume();
    Console.WriteLine($"After Resume(), IsAlive: {thread1.IsAlive},
ThreadState: {thread1.ThreadState}");
    // thread1 is in Abort state
    //In this case, it will start the termination, IsAlive still {
true
    thread1.Abort();
    Console.WriteLine($"After Abort(), IsAlive: {thread1.IsAlive},
ThreadState: {thread1.ThreadState}");
    //Calling the Start Method on a dead thread will result an Exce
    thread1.Start();
}
catch (Exception ex)
{
    Console.WriteLine($"Exception Occurred: {ex.Message}");
}

Console.ReadKey();
}
public static void SomeMethod()
{
    for (int x = 0; x < 3; x++)
    {
        Thread.Sleep(1000);
    }
    Console.WriteLine("SomeMethod Completed...");
}
}
}

```

Output:

```
Before Start, IsAlive: False, ThreadState: Unstarted  
After Start(), IsAlive: True, ThreadState: Running  
After Suspend(), IsAlive: True, ThreadState: SuspendRequested  
After Resume(), IsAlive: True, ThreadState: Running  
After Abort(), IsAlive: True, ThreadState: AbortRequested  
Exception Occurred: Thread is running or terminated; it cannot restart.
```

In the next article, I am going to discuss [Thread Priority in C#](#) with Examples. Here, in this article, I try to explain **Thread Life Cycle in C#** with Examples. I hope you enjoy this Thread Life Cycle in C# with Examples article.